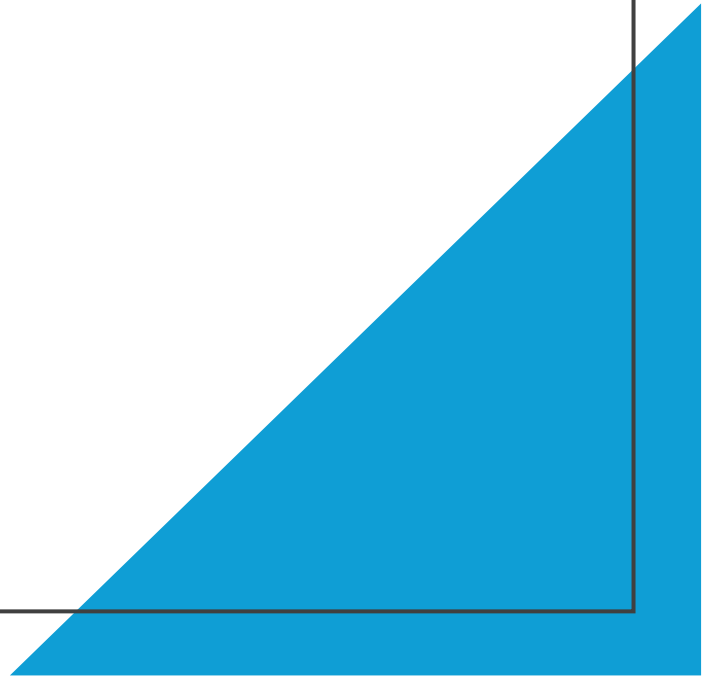


Basic Performance Engineering

Xiaoyuan Suo



How Do We Evaluate Performance? (Big Picture)

- Key questions
 - Is it faster than the serial version?
 - How well does it use more cores?
 - What limits further speedup?
 - Metrics we care about
- Execution time
 - Speedup
 - Efficiency
 - Scalability
- Performance engineering =
 - measure → analyze → explain → improve

Speedup and Efficiency

- Speedup

- Speedup $S(p) = \frac{T_1}{T_p}$
 - T_1 :runtime on 1 core
 - T_p :runtime on p cores
 - Ideal: $S(p) = p$

- Efficiency

- Efficiency $E(p) = \frac{S(p)}{p}$
 - Measures resource utilization
 - $E(p) = 1$:perfect scaling
 - Low efficiency $\rightarrow$ overhead, contention, or serial work

- Python reality

- Overhead matters (process startup, IPC, GIL, memory traffic)

Scaling Laws: Amdahl vs. Gustafson

- Amdahl's Law (Strong Scaling)

- $S(p) = \frac{1}{(1-f) + \frac{f}{p}}$

- f :parallel fraction
 - Fixed problem size
 - Shows hard upper limit

- Key message: Serial parts dominate at large core counts

- Gustafson's Law (Weak Scaling)

- $S(p) = p - (1 - f)(p - 1)$

- Problem size grows with cores
 - More realistic for HPC & data workloads

- Key message: Scale the problem, not just the cores

Why These Laws Matter in Python

- Python-specific constraints
 - GIL limits threading for CPU-bound code
 - Multiprocessing adds overhead
 - Memory bandwidth becomes a bottleneck quickly
- Interpretation
 - Amdahl explains why speedup saturates
 - Gustafson explains why large datasets still scale
- Performance $\neq$ just “use more cores”

Measuring Performance Correctly in Python

- Timing tools
 - `time.perf_counter()` (preferred)
 - `timeit`
 - Wall-clock vs CPU time
- Best practices
 - Multiple runs (median-of-3)
 - Warm-up runs
 - Same input, same environment
- Common student mistake
 - “I timed it once and it was faster.” -- run multiple times

Example (Single Run)

```
def work(n):  
    s = 0  
    for i in range(n):  
        s += i*i  
    return s
```

```
#time.perf_counter()  
#Highest available resolution  
#Includes all overhead (sleep, I/O, scheduling)  
#Best for end-to-end performance
```

```
import time
```

```
start = time.perf_counter()  
work(10_000_000)  
end = time.perf_counter()
```

```
print(f"Elapsed time: {end - start:.4f} seconds")
```

Example (Multiple Run)

#There may be problems with a single run.

```
times = []
```

```
for _ in range(3):
```

```
    start = time.perf_counter()
```

```
    work(10_000_000)
```

```
    times.append(time.perf_counter() - start)
```

```
print(f"Median time: {sorted(times)[1]:.4f} seconds")
```


Example (Microbenchmarking)

```
#use timeit
#Automatically repeats
#Reduces timing noise
#Not ideal for large parallel programs

import timeit

t = timeit.timeit(
    stmt="work(1_000_000)",
    globals=globals(),
    number=5
)

print(f"Average time: {t / 5:.4f} seconds")
```

Example (Wall-Clock Time vs CPU Time)

Wall clock

- #What users actually experience
- #Affected by OS, I/O, waiting
- # Wall-clock time decreases with parallelism

```
import time
```

```
start = time.perf_counter()  
work(10_000_000)  
print("Wall time:", time.perf_counter() - start)
```

CPU time

- #Time spent executing on CPU
- #Ignores sleep and I/O
- # CPU time may increase due to overhead

```
import time
```

```
start = time.process_time()  
work(10_000_000)  
print("CPU time:", time.process_time() - start)
```

Avoid mistakes

- import time
- ~~start = time.time()~~
 - ~~#~~ **×** low resolution, system dependent
- work(10_000_000)
- print(time.time() - start)

Not monotonic

Discouraged for performance studies

- Use this
 - time.perf_counter() → real performance
 - Multiple runs → median
 - timeit → small benchmarks
- Avoid
 - Single runs
 - time.time() for benchmarking

Baseline First: Serial Performance

- Rule #1 of performance engineering
 - You cannot speed up what you did not measure.
- Why serial matters
 - Sets the upper bound
 - Helps identify algorithmic inefficiencies
 - Often dominates total runtime
- Teaching moment
 - Faster algorithm beats parallelism

Parallel Overhead in Python

- Where overhead comes from
 - Process creation (multiprocessing)
 - Data serialization (pickling)
 - Inter-process communication
 - Synchronization and waiting
- Visible symptoms
 - Speedup < 1
 - Efficiency drops rapidly
 - More cores $\rightarrow$ slower runtime

Parallel Overhead Example

- Too-small tasks: parallel is slower than serial
- overhead (pool + IPC + scheduling) dominates when work per task is tiny.

```
import time, multiprocessing as mp
```

```
def tiny_task(x):  
    return x * x # very small work
```

```
def run_serial(n=200_000):  
    start = time.perf_counter()  
    out = [tiny_task(i) for i in range(n)]  
    return time.perf_counter() - start
```

```
def run_parallel(n=200_000, procs=4, chunksize=1):  
    start = time.perf_counter()  
    with mp.Pool(processes=procs) as pool:  
        out = pool.map(tiny_task, range(n), chunksize=chunksize)  
    return time.perf_counter() - start
```

```
if __name__ == "__main__":  
    t1 = run_serial()  
    t4 = run_parallel(procs=4, chunksize=1)  
    print("serial:", t1)  
    print("parallel(4):", t4)
```

Parallel Overhead Example

- Chunk size reduces overhead (same computation)
- Bigger chunksize lowers dispatch overhead (often dramatically).

```
import time, multiprocessing as mp

def tiny_task(x):
    return x * x

def run_parallel(n=200_000, procs=4, chunksize=1):
    start = time.perf_counter()
    with mp.Pool(processes=procs) as pool:
        pool.map(tiny_task, range(n), chunksize=chunksize)
    return time.perf_counter() - start

if __name__ == "__main__":
    for cs in [1, 10, 100, 1000, 10_000]:
        t = run_parallel(chunksize=cs)
        print(f"chunksize={cs:>5} time={t:.4f}s")
```

Parallel Overhead Example

- Process startup overhead (spawn + warmup effect)
- creating a Pool repeatedly costs time; reuse pools in real workflows.

```
import time, multiprocessing as mp

def work(_):
    s = 0
    for i in range(200_000):
        s += i*i
    return s

def one_call(procs=4):
    start = time.perf_counter()
    with mp.Pool(procs) as pool:
        pool.map(work, range(procs)) # one task per process
    return time.perf_counter() - start

if __name__ == "__main__":
    for trial in range(3):
        print("trial", trial, "time", one_call())
```


Parallel Overhead Example

- Pickling / data transfer overhead (sending big objects)
- multiprocessing must serialize arguments → big objects = big overhead.

```
import time, multiprocessing as mp

def f(arr):
    return arr[0] + arr[-1] # trivial compute

if __name__ == "__main__":
    big = list(range(5_000_000)) # large object to serialize

    start = time.perf_counter()
    with mp.Pool(4) as pool:
        pool.map(f, [big] * 8) # sends 'big' 8 times (pickling cost!)
    print("time:", time.perf_counter() - start)
```

Parallel Overhead Example

- Fix pickling overhead using globals + initializer
- send small inputs; keep large read-only data in worker state.

```
import time, multiprocessing as mp
```

```
BIG = None
```

```
def init_worker(shared):
```

```
    global BIG
```

```
    BIG = shared # each process keeps its own reference
```

```
def f(i):
```

```
    return BIG[0] + BIG[-1] + i
```

```
if __name__ == "__main__":
```

```
    big = list(range(5_000_000))
```

```
    start = time.perf_counter()
```

```
    with mp.Pool(4, initializer=init_worker, initargs=(big,)) as pool:
```

```
        pool.map(f, range(8)) # send only small ints now
```

```
    print("time:", time.perf_counter() - start)
```

Parallel Overhead Example

- Over-parallelization: more processes can hurt
- too many processes → context switching + memory contention → slower.
- Lab (parallel_overhead_lab):
Naïve slower because it pays:
 - task scheduling + IPC + pickling big data.
- Optimized faster because it reduces:
 - # of tasks
 - messages between processes
 - data movement (big list sent once per worker, not per task)

```
import os, time, multiprocessing as mp
```

```
def cpu_work(n):  
    s = 0  
    for i in range(n):  
        s += i*i  
    return s
```

```
def bench(procs, tasks=16, n=2_000_000):  
    start = time.perf_counter()  
    with mp.Pool(procs) as pool:  
        pool.map(cpu_work, [n]*tasks, chunksize=1)  
    return time.perf_counter() - start
```

```
if __name__ == "__main__":  
    cpu = os.cpu_count()  
    for p in [1, 2, 4, 8, cpu, cpu*2]:  
        t = bench(p)  
        print(f"procs={p:<3} time={t:.3f}s")
```

Parallel Overhead in Python — BAD vs GOOD (with Speedup)

 BAD: Fine-grained tasks (high overhead)

```
import time, multiprocessing as mp

def work(x):
    return x * x

data = range(200_000)

t0 = time.perf_counter()
with mp.Pool(4) as p:
    p.map(work, data, chunksize=1)
    # too many tasks
t_bad = time.perf_counter() - t0
```

 GOOD: Coarse-grained tasks (lower overhead)

```
t0 = time.perf_counter()
with mp.Pool(4) as p:
    p.map(work, data, chunksize=10_000)
t_good = time.perf_counter() - t0
```

Serial Baseline + Speedup

```
t0 = time.perf_counter()
[work(x) for x in data]
t_serial = time.perf_counter() - t0

print("Speedup (bad): ", t_serial / t_bad)
print("Speedup (good):", t_serial / t_good)
```

Parallelism fails when overhead dominates—coarse tasks recover speedup.

Strong Scaling vs Weak Scaling (Python View)

- Strong Scaling
 - Fixed input size
 - Increase cores
 - Shows overhead and serial limits
- Weak Scaling
 - Increase input with cores
 - Shows algorithm scalability
- Python takeaway
 - Strong scaling breaks early
 - Weak scaling is often more realistic

Strong Scaling vs Weak Scaling Example

- Naïve Parallel Is Slower Than Serial (Overhead Dominates)
- Task scheduling + IPC overhead outweighs computation → parallel is slower.

```
import time, multiprocessing as mp

def work(x):
    return x * x # very small task

if __name__ == "__main__":
    data = range(200_000)

    # Serial
    t0 = time.perf_counter()
    [work(x) for x in data]
    print("serial:", time.perf_counter() - t0)

    # Parallel (naïve)
    t0 = time.perf_counter()
    with mp.Pool(4) as p:
        p.map(work, data, chunksize=1)
    print("parallel:", time.perf_counter() - t0)
```

Strong Scaling vs Weak Scaling Example

- Fixing Overhead with Larger Chunks
- Increasing chunk size reduces scheduling overhead and improves performance.

```
import time, multiprocessing as mp

def work(x):
    return x * x

if __name__ == "__main__":
    data = range(200_000)

    for cs in [1, 100, 10_000]:
        t0 = time.perf_counter()
        with mp.Pool(4) as p:
            p.map(work, data, chunksize=cs)
        print(f"chunksize={cs:>5}
time={time.perf_counter()-t0:.4f}")
```

Compute-Bound vs Memory-Bound

- Compute-bound
 - Heavy arithmetic
 - Scales better with cores
 - Meaning: runtime is dominated by arithmetic/CPU instructions. If you make the CPU do more math per byte loaded, it gets slower. More cores often help.
- Memory-bound
 - Large arrays
 - Cache misses
 - Bandwidth limited
 - Meaning: runtime is dominated by moving data (RAM → cache → CPU). The CPU is often waiting on memory. Adding more cores may not help much (bandwidth saturates).
- Python often is
 - Memory-bound before compute-bound
- A practical way to think: arithmetic intensity = (operations) / (bytes moved).
Low arithmetic intensity → memory-bound. High → compute-bound.

Memory-bound vs Compute-bound toggle

- `np.sum(a)` streams memory once → memory-bound.
- The loop does many FLOPs per element → becomes more compute-bound.

```
import numpy as np
import time

N = 30_000_000
a = np.random.rand(N).astype(np.float64)

def time_it(fn, reps=3):
    ts = []
    for _ in range(reps):
        t0 = time.perf_counter()
        fn()
        ts.append(time.perf_counter() - t0)
    return sorted(ts)[1]

# Memory-bound: single pass over a (mostly bandwidth)
def mem_bound():
    return np.sum(a)

# More compute per element: extra arithmetic work (higher intensity)
def compute_bound():
    x = a
    for _ in range(20):
        x = x * 1.0000001 + 0.0000003
    return np.sum(x)

tm = time_it(mem_bound)
tc = time_it(compute_bound)

print(f"memory-bound sum:      {tm:.4f}s")
print(f"compute-heavier loop:    {tc:.4f}s")
```

Performance Visualization

- What to plot
 - Runtime vs cores
 - Speedup vs cores
 - Efficiency vs cores
- Why plots matter
 - Numbers hide trends
 - Plots reveal saturation points

Numbers tell you what happened; plots tell you why.

Visualization Example (run time vs. cores)

```
Performance Plot.py - C:/Users/mshgo/Downloads/Performance Plot.py (3.11.9)
File Edit Format Run Options Window Help

import multiprocessing as mp
import time
import matplotlib.pyplot as plt

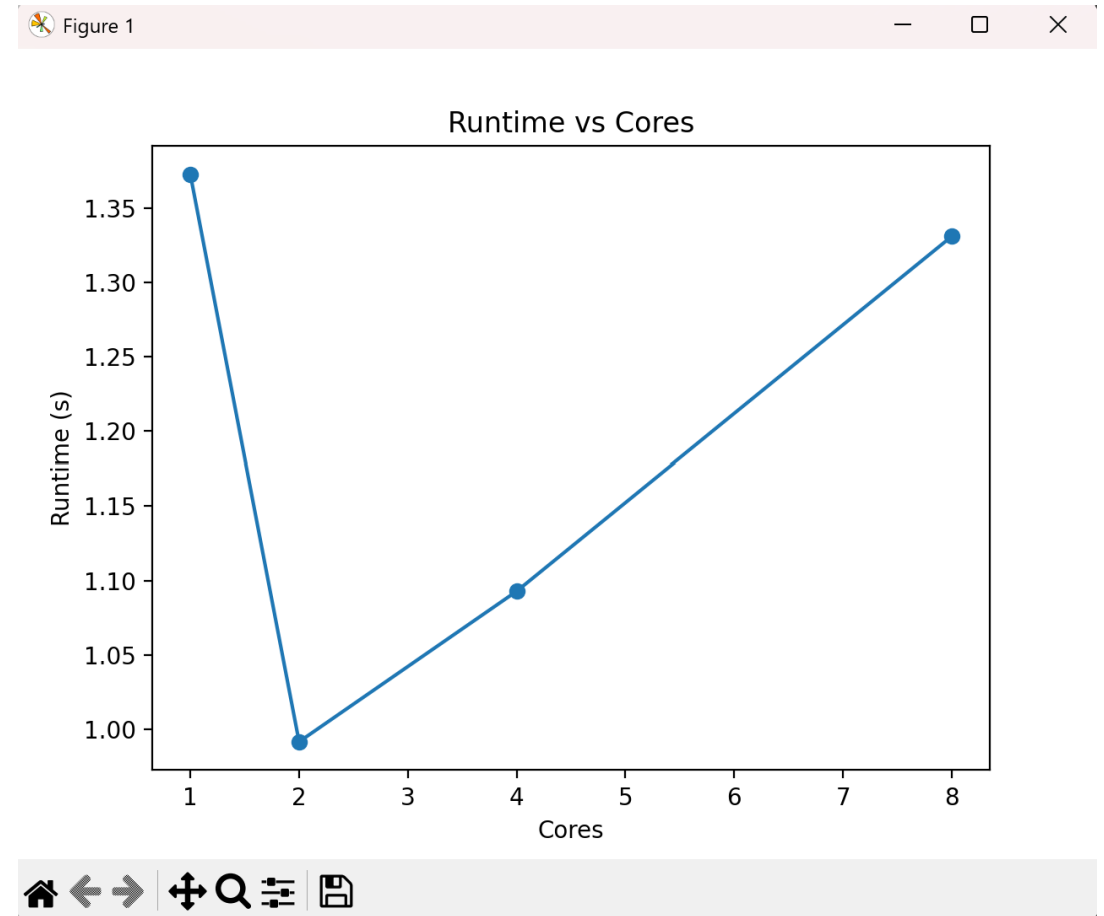
def work(n):
    s = 0
    for i in range(n):
        s += i*i
    return s

def run(p, tasks=8, n=2_000_000):
    start = time.perf_counter()
    with mp.Pool(p) as pool:
        pool.map(work, [n] * tasks, chunksize=1)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8]
    times = []

    for p in cores:
        t = run(p)
        times.append(t)
        print(f"cores={p} time={t:.3f}")

    # Metrics
    t1 = times[0]
    speedup = [t1 / t for t in times]
    efficiency = [s / p for s, p in zip(speedup, cores)]
```



Runtime improves, but less with each added core—overhead grows.

Visualization Example (Speedup vs Cores)

```
Performance Plot.py - C:/Users/mshgo/Downloads/Performance Plot.py (3.11.9)
File Edit Format Run Options Window Help

import multiprocessing as mp
import time
import matplotlib.pyplot as plt

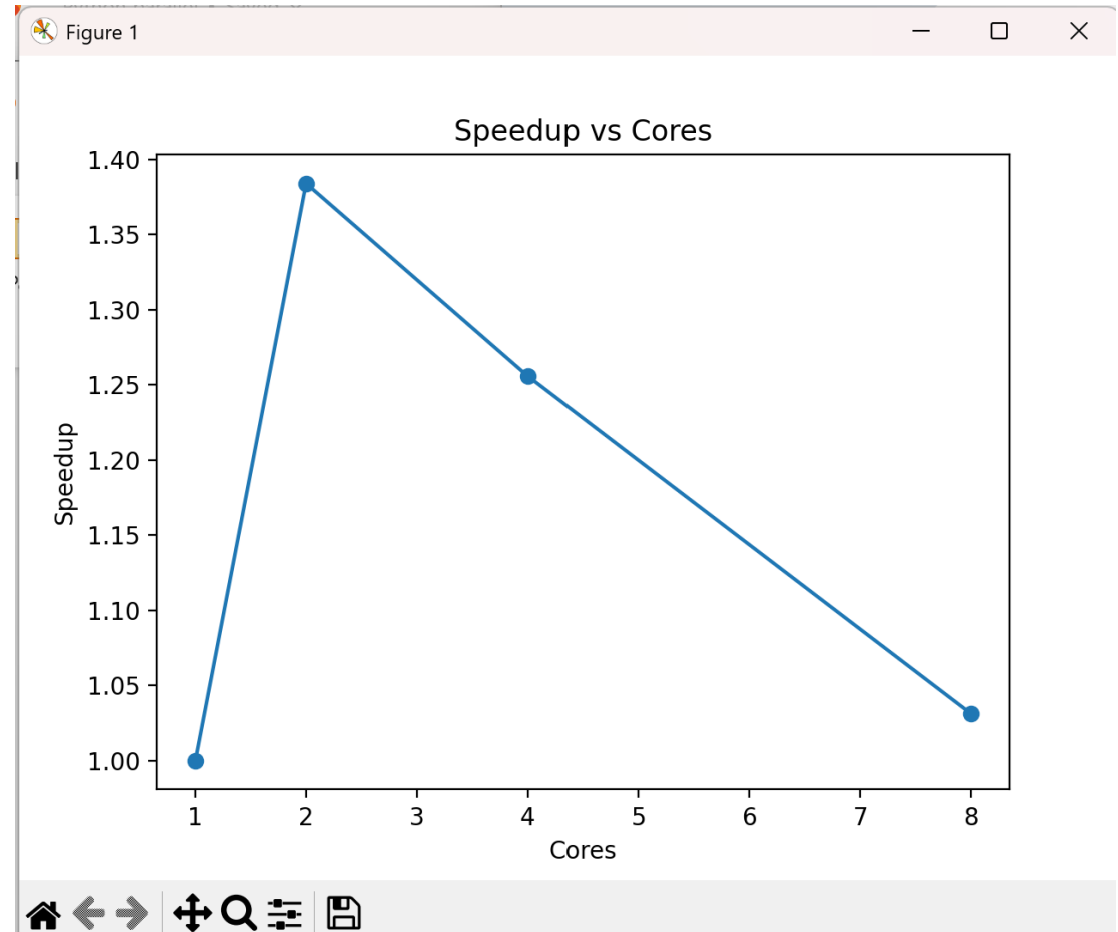
def work(n):
    s = 0
    for i in range(n):
        s += i*i
    return s

def run(p, tasks=8, n=2_000_000):
    start = time.perf_counter()
    with mp.Pool(p) as pool:
        pool.map(work, [n] * tasks, chunksize=1)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8]
    times = []

    for p in cores:
        t = run(p)
        times.append(t)
        print(f"cores={p} time={t:.3f}")

    # Metrics
    t1 = times[0]
    speedup = [t1 / t for t in times]
    efficiency = [s / p for s, p in zip(speedup, cores)]
```



This flattening is Amdahl's Law and parallel overhead in action.

Visualization Example (Efficiency vs Cores)

```
Performance Plot.py - C:/Users/mshgo/Downloads/Performance Plot.py (3.11.9)
File Edit Format Run Options Window Help

import multiprocessing as mp
import time
import matplotlib.pyplot as plt

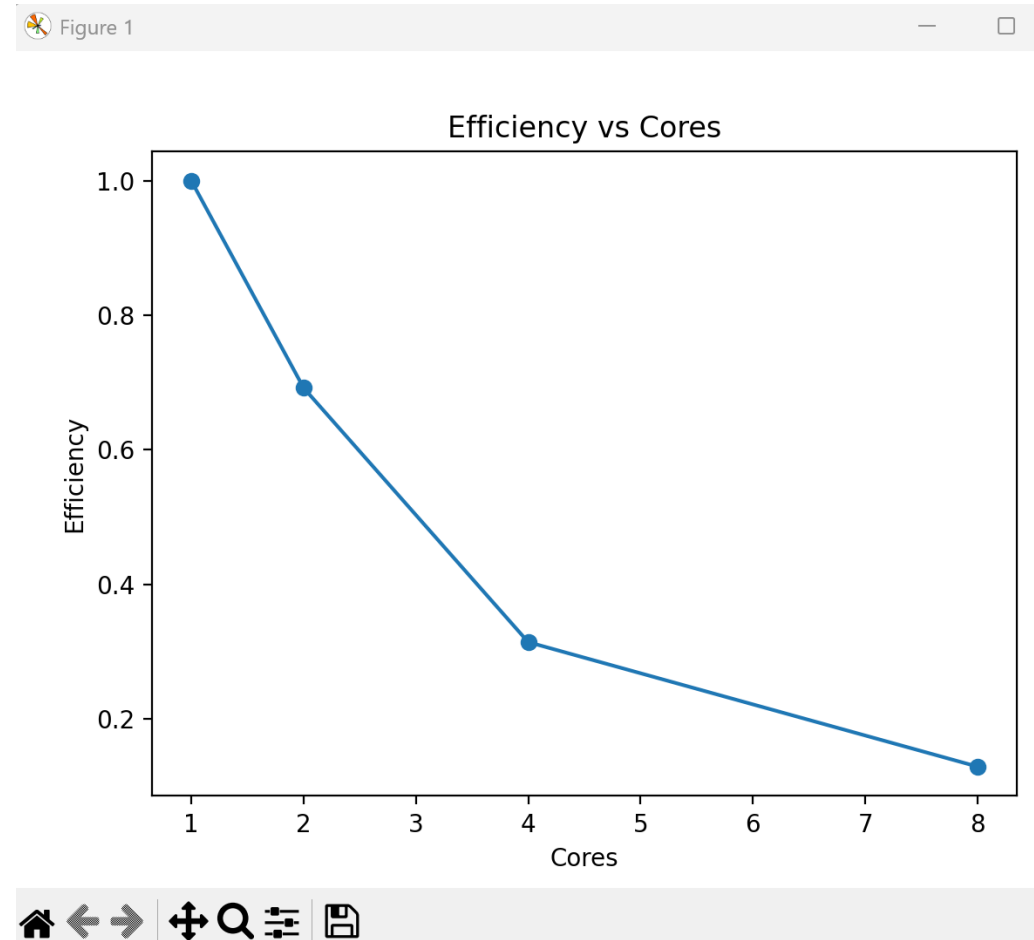
def work(n):
    s = 0
    for i in range(n):
        s += i*i
    return s

def run(p, tasks=8, n=2_000_000):
    start = time.perf_counter()
    with mp.Pool(p) as pool:
        pool.map(work, [n] * tasks, chunksize=1)
    return time.perf_counter() - start

if __name__ == "__main__":
    cores = [1, 2, 4, 8]
    times = []

    for p in cores:
        t = run(p)
        times.append(t)
        print(f"cores={p} time={t:.3f}")

# Metrics
t1 = times[0]
speedup = [t1 / t for t in times]
efficiency = [s / p for s, p in zip(speedup, cores)]
```



We're paying for cores we can't fully use.

NUMA Effects (What it really means)

- NUMA = Non-Uniform Memory Access
 - On modern multi-socket systems, each CPU socket has its own local memory
 - Accessing local memory is fast
 - Accessing remote memory (owned by another socket) is slower
 - The OS tries to help, but your program's memory placement still matters
- Key rule (first-touch policy):
 - Memory is allocated on the NUMA node of the core that first writes to it.
 - This is why parallel programs can slow down when you add more cores.

Why NUMA matters for Python

- Even in Python:
 - multiprocessing spreads processes across sockets
 - Large arrays (NumPy, lists) may live on one NUMA node
 - Other processes access that memory remotely → bandwidth + latency penalty
- Symptom you'll see
 - Runtime stops improving (or gets worse) after some core count
 - CPU utilization looks “high” but performance stalls

NUMA example

- Demonstrate numa.py
 -  Bad case: one process initializes data → all memory on one NUMA node
 - One process allocates the entire array
 - Memory lives on one NUMA node
 - Other processes access it remotely
 - More cores → more contention
 -  Better case: each worker initializes its own chunk (first-touch locality)
 - Each worker allocates its own chunk
 - Memory is local to the worker's socket
 - Better bandwidth utilization
 - Better scaling

NUMA Conclusion

- Memory locality matters as much as parallelism
- First-touch determines where memory lives
- Poor memory placement can destroy scaling
- Fix by:
 - initializing data in workers
 - partitioning data per process
 - avoiding shared large arrays
- This is not a Python problem — it's an architecture problem.
 - NUMA explains why you see:
 - Speedup flattens before full core count
 - Efficiency drops sharply at socket boundaries
 - Runtime improves until memory bandwidth saturates

Interpreting Results (Engineering Mindset)

- When performance is bad, ask:
 - Is this Amdahl's Law?
 - Is overhead dominating?
 - Is memory the bottleneck?
 - Is the algorithm appropriate?
- Key habit
 - Don't guess. Measure and explain.

Typical Student Pitfalls

- No serial baseline
- Timing includes I/O
- Too small problem sizes
- Assuming linear speedup
- Ignoring variability

Where to Go Next

- NUMA effects
- Thread affinity
- Vectorization (NumPy)
- Hybrid models (MPI + Python)
- Profiling tools (cProfile, perf, htop)
- Python + C/C++ extensions

Takeaway

- Performance engineering is not about making code fast. It's about understanding why it behaves the way it does.